



**VIT<sup>®</sup>**

Vellore Institute of Technology  
(Deemed to be University under section 3 of UGC Act, 1956)

# BCSE204L- Design and Analysis of Algorithms

**Dr. Iyappan Perumal**

Assistant Professor Senior Grade 2

School of Computer Science & Engineering

VIT,Vellore.



## Course Objective

1. To provide mathematical foundations for analysing the complexity of the algorithms
2. To impart the knowledge on various design strategies that can help in solving the real world problems effectively
3. To synthesize efficient algorithms in various engineering design situations

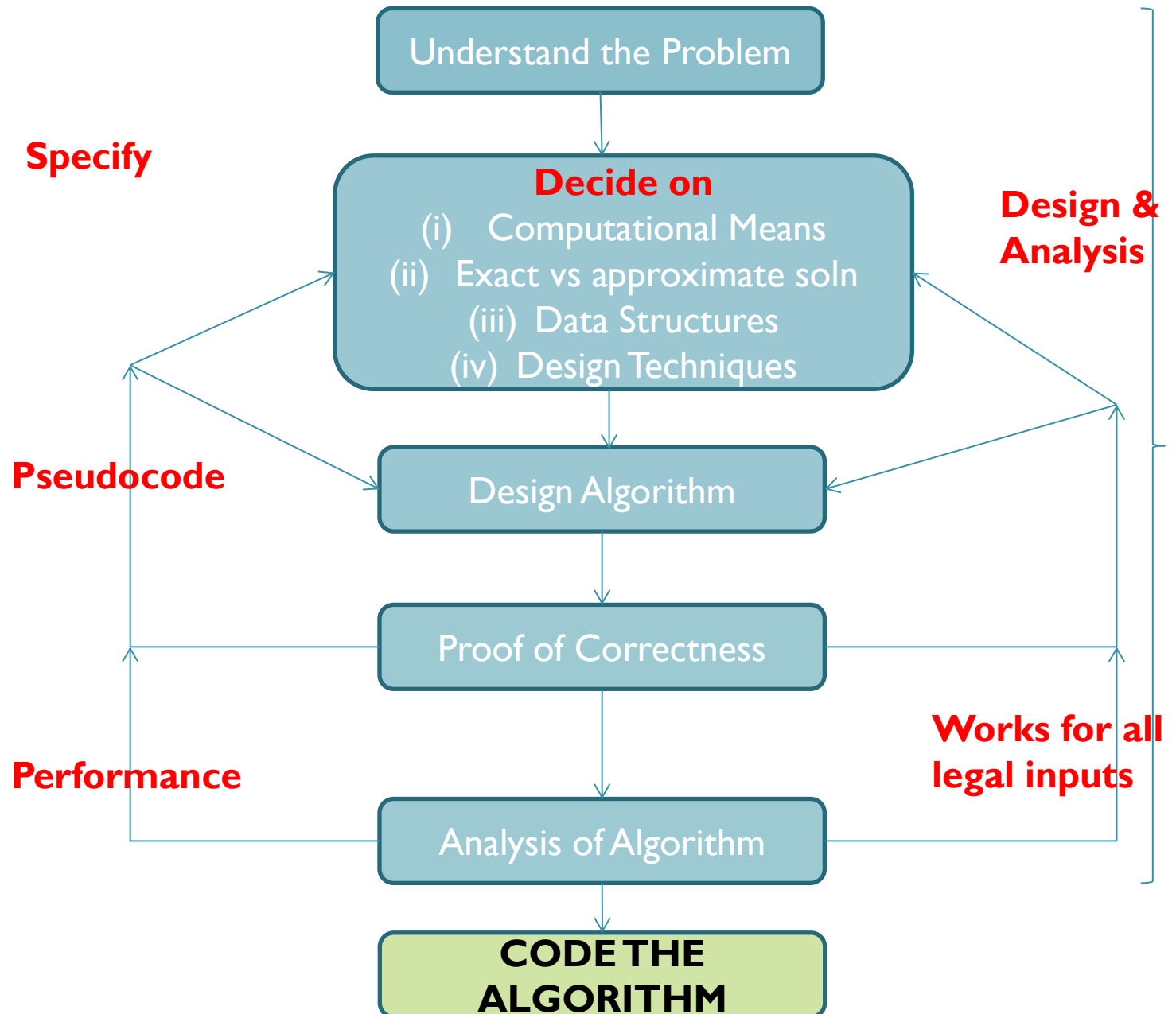


# BCSE204L- Design and Analysis of Algorithms- Course outcome

1. Apply the mathematical tools to analyse and derive the running time of the algorithms
2. Demonstrate the major algorithm design paradigms.
3. Explain major graph algorithms, string matching and geometric algorithms along with their analysis.
4. Articulating Randomized Algorithms.
5. Explain the hardness of real-world problems with respect to algorithmic efficiency and learning to cope with it.



# Algorithm Design & Analysis - Overview



# BCSE204L- Design and Analysis of Algorithms

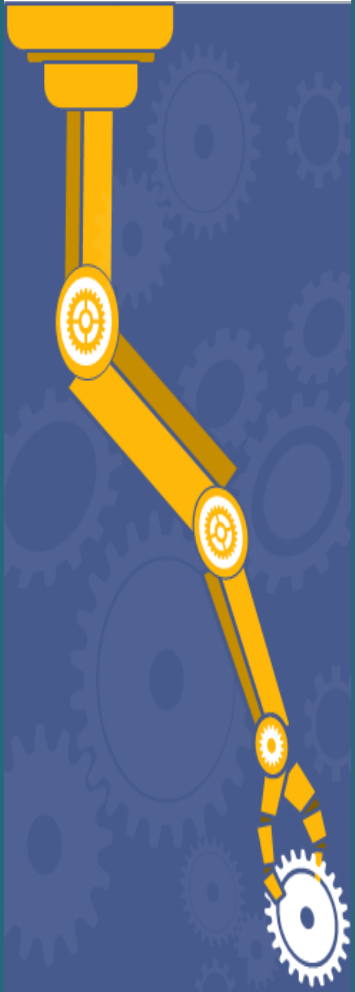
## Assessment Configuration

- Digital Assignment - I
- Quiz-I
- Digital Assignment-2/Quiz-2
- CAT-I
- CAT-2
- FAT



## Lab Component Assessment

- Assessment questions will be uploaded in VTOP/ intimated in class.
- An assessment will include questions from multiple lab sessions
- Programming type (C,C++)
- Attach the screenshot of the output after the code.
- Check the output for multiple and diverse test cases and include all screenshots.
- Only a single PDF document is allowed per assessment.
- Submission can be done only through VTOP



## General Guidelines

- All DAs must be submitted through VTOP only.
- No other mode (such as email) is accepted
- Submit assignments well in advance to avoid probable network issues/VTOP login errors at the last moment.

**Queries????**

**All the Best**





# BCSE204L- Design and Analysis of Algorithms- Modules

- Module-1: Design Paradigms: Greedy, Divide and Conquer Techniques
- Module-2: Design Paradigms: Dynamic Programming, Backtracking and Branch & Bound Techniques
- Module-3: String Matching algorithms
- Module-4: Graph Algorithms
- Module-5: Geometric Algorithms
- Module-6: Randomized Algorithms
- Module-7: Classes of Complexity and Approximation Algorithms.





# BCSE204L- Design and Analysis of Algorithms- Syllabus

|                                                                                                                                                                                                                                                                                                                                                                                                                         |                                                                                              |                 |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-----------------|
| <b>Module:1</b>                                                                                                                                                                                                                                                                                                                                                                                                         | <b>Design Paradigms: Greedy, Divide and Conquer Techniques</b>                               | <b>6 hours</b>  |
| Overview and Importance of Algorithms - Stages of algorithm development: Describing the problem, Identifying a suitable technique, Design of an algorithm, Derive Time Complexity, Proof of Correctness of the algorithm, Illustration of Design Stages - Greedy techniques: Fractional Knapsack Problem, and Huffman coding - Divide and Conquer: Maximum Subarray, Karatsuba faster integer multiplication algorithm. |                                                                                              |                 |
| <b>Module:2</b>                                                                                                                                                                                                                                                                                                                                                                                                         | <b>Design Paradigms: Dynamic Programming, Backtracking and Branch &amp; Bound Techniques</b> | <b>10 hours</b> |
| Dynamic programming: Assembly Line Scheduling, Matrix Chain Multiplication, Longest Common Subsequence, 0-1 Knapsack, TSP- Backtracking: N-Queens problem, Subset Sum, Graph Coloring- Branch & Bound: LIFO-BB and FIFO BB methods: Job Selection problem, 0-1 Knapsack Problem                                                                                                                                         |                                                                                              |                 |
| <b>Module:3</b>                                                                                                                                                                                                                                                                                                                                                                                                         | <b>String Matching Algorithms</b>                                                            | <b>5 hours</b>  |
| Naïve String-matching Algorithms, KMP algorithm, Rabin-Karp Algorithm, Suffix Trees.                                                                                                                                                                                                                                                                                                                                    |                                                                                              |                 |
| <b>Module:4</b>                                                                                                                                                                                                                                                                                                                                                                                                         | <b>Graph Algorithms</b>                                                                      | <b>6 hours</b>  |
| All pair shortest path: Bellman Ford Algorithm, Floyd-Warshall Algorithm - Network Flows: Flow Networks, Maximum Flows: Ford-Fulkerson, Edmond-Karp, Push Re-label Algorithm – Application of Max Flow to maximum matching problem                                                                                                                                                                                      |                                                                                              |                 |
| <b>Module:5</b>                                                                                                                                                                                                                                                                                                                                                                                                         | <b>Geometric Algorithms</b>                                                                  | <b>4 hours</b>  |
| Line Segments: Properties, Intersection, sweeping lines - Convex Hull finding algorithms: Graham's Scan, Jarvis' March Algorithm.                                                                                                                                                                                                                                                                                       |                                                                                              |                 |
| <b>Module:6</b>                                                                                                                                                                                                                                                                                                                                                                                                         | <b>Randomized algorithms</b>                                                                 | <b>5 hours</b>  |
| Randomized quick sort - The hiring problem - Finding the global Minimum Cut.                                                                                                                                                                                                                                                                                                                                            |                                                                                              |                 |
| <b>Module:7</b>                                                                                                                                                                                                                                                                                                                                                                                                         | <b>Classes of Complexity and Approximation Algorithms</b>                                    | <b>7 hours</b>  |
| The Class P - The Class NP - Reducibility and NP-completeness – SAT (Problem Definition and statement), 3SAT, Independent Set, Clique, Approximation Algorithm – Vertex Cover, Set Cover and Travelling salesman                                                                                                                                                                                                        |                                                                                              |                 |
| <b>Module:8</b>                                                                                                                                                                                                                                                                                                                                                                                                         | <b>Contemporary Issues</b>                                                                   | <b>2 hours</b>  |

# BCSE204L- Design and Analysis of Algorithms- References

## Text Books:

1. Thomas H. Cormen, C.E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 2009, 3<sup>rd</sup> Edition, MIT Press.

## Reference Books:

1. Jon Kleinberg, Éva Tardos, Algorithm Design, Pearson education, 2014.
2. Rajeev Motwani, Prabhakar Raghavan; Randomized Algorithms, Cambridge University Press, 1995 (Online Print – 2013)
3. Ravindra K. Ahuja, Thomas L. Magnanti and James B. Orlin, “ Network Flows: Theory, Algorithms and Applications”, Pearson Education, 2014.



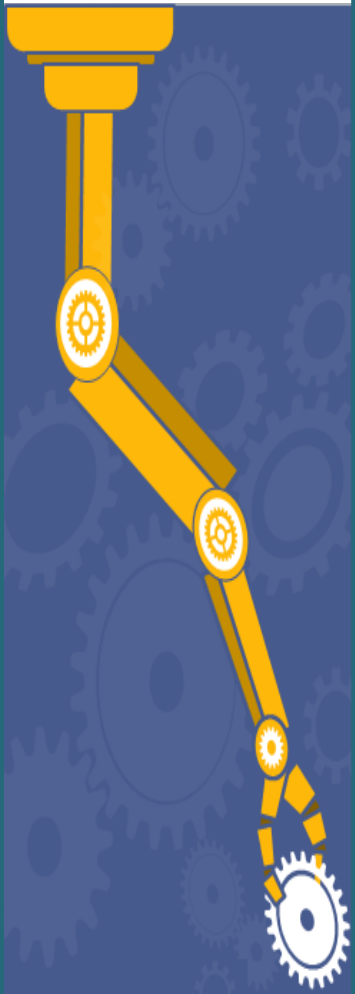
# Module I: Design Paradigms: Greedy, Divide and Conquer Techniques(6 Hours)

- **Overview and Importance of Algorithms**
  - **Stages of algorithm development:**
    - Describing the problem
    - Identifying a suitable technique
    - Design of an algorithm
    - Derive Time Complexity,
    - Proof of Correctness of the algorithm
- **Illustration of Design Stages**
  - **Greedy techniques:**
    - Fractional Knapsack Problem
    - Huffman coding
  - **Divide and Conquer:**
    - Maximum Subarray
    - Karatsuba faster integer multiplication algorithm.



# Algorithm

- **An algorithm is a finite set of instructions or logic, written in order, to accomplish a predefined task.**
  - **Input-** There should be 0 or more inputs supplied externally to the algorithm.
  - **Output-** There should be at least 1 output obtained.
  - **Definiteness-** Every step of the algorithm should be clear and well defined.
  - **Finiteness-** Algorithm should have finite number of steps.
  - **Correctness-** Every step of the algorithm must generate a correct output.
- **An algorithm is said to be efficient and fast, if it takes less time to execute and consumes less memory space.**
  - Time Complexity
  - Space Complexity



# Stages of algorithm development

- Stages of algorithm development for solving a problem
  - Describing the problem.
  - Identifying a suitable technique
  - Design of an algorithm
  - Proof of Correctness of the algorithm.





# Proof of Correctness of the algorithm

- Sorting Problem: Insertion Sort

**Input:** A sequence of  $n$  numbers  $\langle a_1, a_2, \dots, a_n \rangle$ .

**Output:** A permutation (reordering)  $\langle a'_1, a'_2, \dots, a'_n \rangle$  of the input sequence such that  $a'_1 \leq a'_2 \leq \dots \leq a'_n$ .



- Insertion sort works in the way many people sort a hand of playing cards.



# Proof of Correctness of the algorithm

## Insertion Sort Algorithm

Current element which is to be inserted

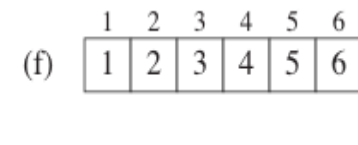
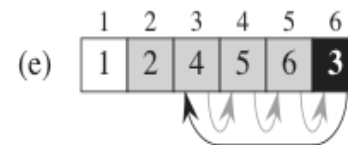
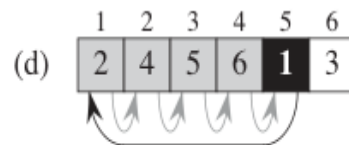
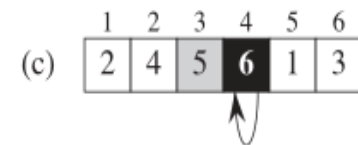
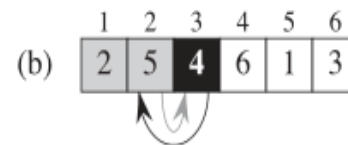
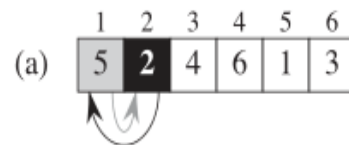
INSERTION-SORT( $A$ )

```

1  for  $j = 2$  to  $A.length$ 
2       $key = A[j]$ 
3      // Insert  $A[j]$  into the sorted sequence  $A[1..j-1]$ .
4       $i = j - 1$ 
5      while  $i > 0$  and  $A[i] > key$ 
6           $A[i+1] = A[i]$ 
7           $i = i - 1$ 
8       $A[i+1] = key$ 
    
```

Sorted Subsequence

$A[j+1 \dots n]$  - remaining numbers

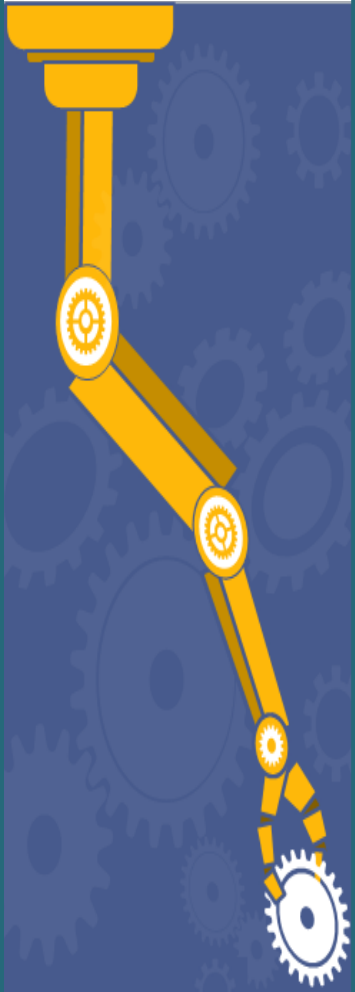




# Proof of Correctness of the algorithm

- **Loop Invariant Method**

- A loop invariant is a condition (among program variables) that is necessarily true immediately before and immediately after each iteration of a loop.
- In other words, it is **some condition** that holds for every iteration of the loop.



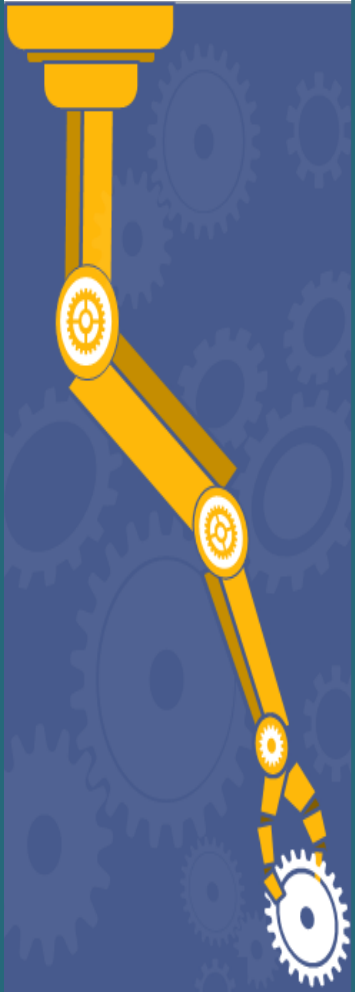
# Proof of Correctness of the algorithm

- **Loop Invariant Method- Basic Example**

- Consider an Array  $a=\{8,6,4,11,3,7\}$  with 6 elements and we have to find maximum element “max” in the array.

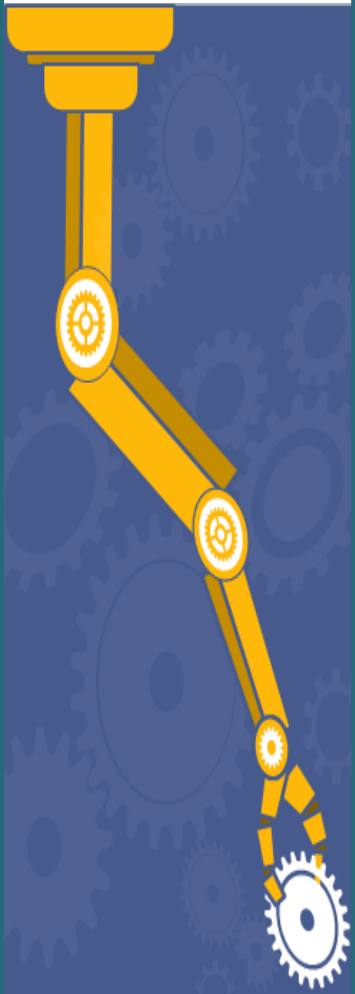
```
Alg max(a[ ],max)
{
    for(i=0;i<6;i++)
    {
        if(a[i]>max)
            max=a[i]
    }
}
```

Here, loop invariant condition is that “Max” is always maximum among the first “i” elements of the array A



# Proof of Correctness of the algorithm

- **Loop Invariant Method**
  - At the start of each iteration of the for loop of lines 1–8, the subarray  $A[1.....j-1]$  consists of the elements originally in  $A[1.....j-1]$  but in sorted order. This Property is formally stated as Loop invariant
- In loop invariant method of proving correctness, one must show three things about a loop invariant.
  - **Initialization:** It is true prior to the first iteration of the loop.
  - **Maintenance:** If it is true before an iteration of the loop, it remains true before the next iteration.
  - **Termination:** When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

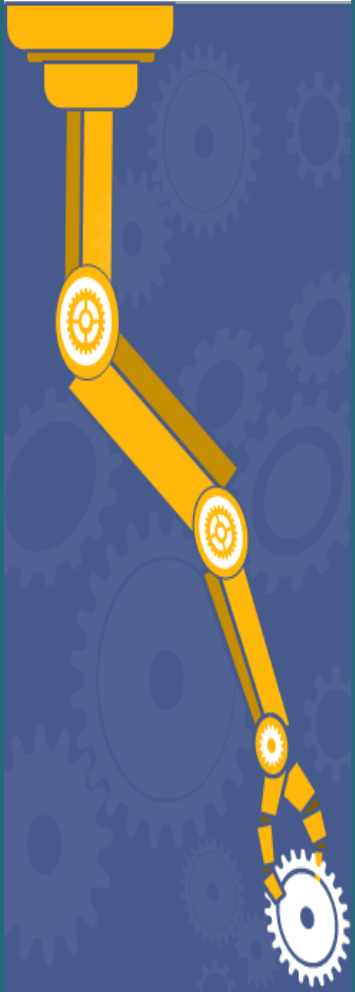


# Proof of Correctness of the algorithm

- **Initialization**

- To show that the loop invariant holds before the first loop iteration
- When  $j = 2$ , the subarray  $A[1..j - 1]$  consists of just the single element  $A[1]$
- Which is the original element in  $A[1]$
- This subarray is trivially sorted

- Shows that Loop invariant holds prior to the first iteration of the loop.



## Proof of Correctness of the algorithm

- **Maintenance** : Each iteration maintains the loop invariant.
- The body of the for loop works by moving  $A[j - 1], A[j - 2], A[j - 3]$ , and so on by one position to the right until it finds the proper position for  $A[j]$  (lines 4–7), at which point it inserts the value of  $A[j]$  (line 8).
- The subarray  $A[1..j]$  then consists of the elements originally in  $A[1..j]$  , but in sorted order.
- Incrementing  $j$  for the next iteration of the for loop then preserves the loop invariant



## Proof of Correctness of the algorithm

- **Termination :What happens when the loop terminates??**
- The condition causing the for loop to terminate is that  $j > A.length = n$
- Substituting  $n+1$  for  $j$  in the wording of loop invariant, we have that the subarray  $A[1..n]$  consists of the elements originally in  $A[1..n]$ , but in sorted order
- As  $A[1..n]$  is the original array it implies that the entire array is sorted
- Hence the algorithm is correct





## Similar Problem-Try Yourself

### Consider the searching problem:

- Input: A sequence of  $n$  numbers  $A = (a_1, a_2, a_3, \dots, a_n)$  and a value “Key”.
- Output: An index  $i$  such that “ $\text{Key} = A[i]$ ” or NIL if does not appear in  $A$ .
- Write pseudocode for linear search, which scans through the sequence, looking for Key?.
- Using a loop invariant, prove that your algorithm is correct. Make sure that your loop invariant fulfils the three necessary properties.

